

Week 7. 최신 import(import) 블록 + generate-config) & GitHub Actions OIDC

개념 오늘의 목표 & 핵심 순간 ★

(1) Week6의 노가다를 없애는 최신 import: import 블록 + terraform plan -generate-config-out 이 HCL 을 자동 생성해준다. (2) CI에서 장기 AccessKey를 심는 대신 OIDC로 임시 자격증명을 받는다 — 왜 장기 키가 위험한지가 핵심.

실습 1. 콘솔에서 버킷 하나 수동 생성

콘솔 → S3 → boaz26-w7-import-kdh1834 생성(리전 ap-northeast-2).

실습 2. ★ import 블록 작성 (명령어 대신 코드로)

week7/import.tf :

```
import {
  to = aws_s3_bucket.w7
  id = "boaz26-w7-import-kdh1834"
}
```

week7/main.tf (provider만):

```
terraform {
  required_version = ">= 1.9"
  required_providers { aws = { source = "hashicorp/aws", version = "~> 6.0" } }
}
provider "aws" { region = "ap-northeast-2" }
```

실습 3. ★ config 자동 생성

```
cd week7 && terraform init
terraform plan -generate-config-out=generated.tf
```

확인 HCL 자동 생성

generated.tf 가 만들어지고, 그 안에 resource "aws_s3_bucket" "w7" { ... } 가 실제 속성으로 자동 채워져 있다. (Week6의 손 노가다가 사라짐 ★)

실습 4. import 반영

```
terraform apply      # import 실행 + state 편입
terraform plan       # "No changes" 여야 정상
```

관찰 🍷 고전 vs 최신 비교

	Week6 고전	Week7 최신
명령	<code>terraform import</code>	<code>import</code> 블록 + <code>apply</code>
HCL 작성	손으로 (노가다)	<code>-generate-config-out</code> 자동

- 체감상 뭐가 더 편했나 한 줄: ____

함정 generate-config 주의

- 생성된 `generated.tf` 는 초안. 리뷰 후 `import` 블록은 apply 뒤 지워도 됨(1회성).
- data source나 이미 관리 중인 리소스가 있으면 생성 안 됨.

개념 ★ 왜 OIDC인가 (장기 키 vs 임시 자격)

CI(GitHub Actions)에서 AWS를 쓰려면 자격증명이 필요하다.

- 장기 AccessKey를 Secret에 저장 = 유출되면 무기한 악용, 순환(rotation) 관리 부담, "누가 썼는지" 추적 어려움.
- OIDC: GitHub이 워크플로 실행마다 서명된 단기 토큰을 발급 → AWS가 이를 신뢰해 몇 분짜리 임시 크레덴셜 부여. 저장할 비밀키가 아예 없음.

```
[GitHub Actions] --OIDC 토큰--> [AWS IAM OIDC Provider]
                                | AssumeRoleWithWebIdentity
                                ▼
                                임시 자격 (15분~1시간)
```

실습 5. AWS 측 — OIDC provider + Role (콘솔/CLI 택1, 개념 확인용)

IAM → Identity providers → Add provider → **OpenID Connect**

- Provider URL: `https://token.actions.githubusercontent.com`
- Audience: `sts.amazonaws.com`

그 다음 Role 생성 — 신뢰 정책(trust policy):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Federated": "arn:aws:iam::<ACCOUNT_ID>:oidc-provider/token.actions.github",
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": { "token.actions.githubusercontent.com:aud": "sts.amazonaws.com" },
      "StringLike": { "token.actions.githubusercontent.com:sub": "repo:<ORG>/<REPO>:ref:refs/"
    }
  }]
}
```

실습 6. GitHub Actions 워크플로 (키 없음)

`.github/workflows/terraform.yml`:

```
name: terraform-plan
on: [push]
permissions:
  id-token: write      # OIDC 토큰 발급에 필수 ★
  contents: read
jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::<ACCOUNT_ID>:role/boaz-gha-oidc
          aws-region: ap-northeast-2
      - uses: hashicorp/setup-terraform@v3
      - run: terraform init
      - run: terraform plan
```

확인 OIDC 동작

Actions 로그에서 `configure-aws-credentials` 가 `AssumeRoleWithWebIdentity` 로 임시 크레덴셜을 획득 → `terraform plan` 성공. 어디에도 `AccessKey`를 저장하지 않았다.

관찰 🍷 마무리 회고

- 장기 키 대비 OIDC의 장점 2가지: __, __
- 7주 중 가장 기억에 남는 순간: __

정리 최종 destroy + 스터디 마무리 체크리스트

```
cd week7 && terraform destroy      # import한 w7 버킷 삭제
```

- ☐ w7 버킷 삭제 확인
- ☐ 실습용 IAM Role / OIDC provider는 남겨도 무해하나, 정리하려면 콘솔에서 삭제
- ☐ 모든 주차 리소스가 계정에서 사라졌는지 최종 점검(요금 0원)
- ☐ 회고: IaC 3대 명령, state의 의미, drift, import, OIDC를 한 문장씩 설명할 수 있으면 수료 🎓